

MANUAL DE USUARIO

Laboratorio 2: Árbol Binario de Búsqueda (BST) en C#

Estudiante: Kimberly. Jimena Escobar Pacheco

Curso: Procesos y algoritmos II

Docente: Ing. Ivan Canel

Fecha: 22/8

Universidad Regional de Guatemala.



1. Introducción

El presente manual describe el funcionamiento de una aplicación de consola desarrollada en C# para administrar un Árbol Binario de Búsqueda (BST).

El programa permite almacenar números enteros y organizarlos de acuerdo con las reglas de un árbol binario de búsqueda. También permite buscar valores y realizar los recorridos Inorden, Preorden y Postorden.

La aplicación fue desarrollada utilizando programación orientada a objetos y métodos recursivos para trabajar directamente sobre la estructura del árbol.

2. Objetivo

Implementar un Árbol Binario de Búsqueda en C# que permita insertar y buscar valores enteros, así como realizar diferentes recorridos sobre los nodos almacenados.

3. Requisitos para ejecutar el programa

Para ejecutar el programa se necesita:

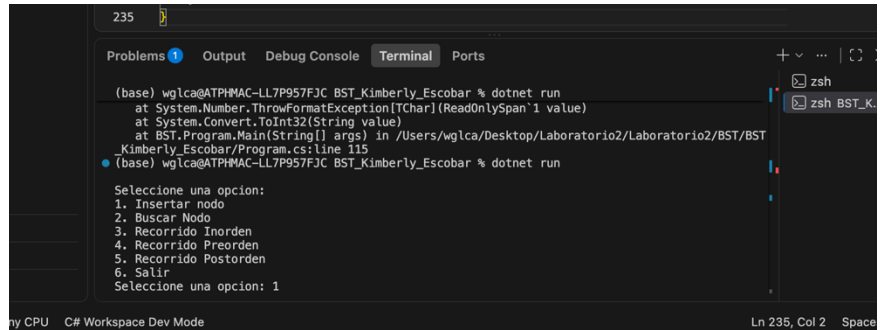
- Computadora.
- .NET SDK instalado.
- Visual Studio Code.
- Extensión de C# para Visual Studio Code.

El programa funciona como una aplicación de consola

4. Menú principal

Al ejecutar el programa se presenta un menú con seis opciones:

1. Insertar nodo.
2. Buscar nodo.
3. Recorrido Inorden.
4. Recorrido Preorden.
5. Recorrido Postorden.
6. Salir.



```
235
Problems Output Debug Console Terminal Ports
(base) wglca@ATPMAC-LL7P957FJC BST_Kimberly_Escobar % dotnet run
at System.Number.ThrowFormatException[TChar](ReadOnlySpan`1 value)
at System.Convert.ToInt32(String value)
at BST.Program.Main(String[] args) in /Users/wglca/Desktop/Laboratorio2/Laboratorio2/BST/BST_Kimberly_Escobar/Program.cs:line 115
(base) wglca@ATPMAC-LL7P957FJC BST_Kimberly_Escobar % dotnet run

Seleccione una opción:
1. Insertar nodo
2. Buscar Nodo
3. Recorrido Inorden
4. Recorrido Preorden
5. Recorrido Postorden
6. Salir
Seleccione una opción: 1
```

El usuario puede seleccionar una opción y, después de realizar la operación, regresar nuevamente al menú.

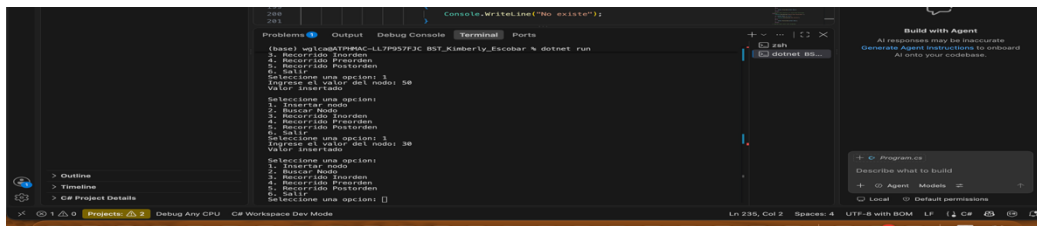
5. Funcionalidades del programa

5.1 Insertar nodo

La opción **1. Insertar nodo** permite ingresar un número entero para almacenarlo en el árbol.

El programa coloca automáticamente el valor en la posición correspondiente siguiendo las reglas del BST:

- Los valores menores se colocan en el lado izquierdo.
- Los valores mayores se colocan en el lado derecho.
- No se permiten valores duplicados.



```
Console.WriteLine("No existe");
(base) wglca@ATPMAC-LL7P957FJC BST_Kimberly_Escobar % dotnet run
1. Insertar nodo
2. Buscar nodo
3. Recorrido Inorden
4. Recorrido Preorden
5. Recorrido Postorden
6. Salir
Seleccione una opción: 1
Ingrese el valor del nodo: 50
Valor Insertado
Seleccione una opción:
1. Insertar nodo
2. Buscar nodo
3. Recorrido Inorden
4. Recorrido Preorden
5. Recorrido Postorden
6. Salir
Seleccione una opción: 1
Ingrese el valor del nodo: 30
Valor Insertado
Seleccione una opción:
1. Insertar nodo
2. Buscar nodo
3. Recorrido Inorden
4. Recorrido Preorden
5. Recorrido Postorden
6. Salir
Seleccione una opción: []
```

En la prueba realizada se ingresaron diferentes valores para comprobar que los nodos se almacenaran correctamente

5.2 Buscar nodo

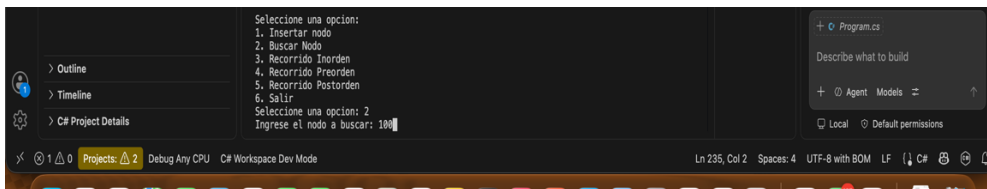
La opción **2. Buscar Nodo** permite ingresar un número para comprobar si se encuentra almacenado dentro del árbol.

Si el valor existe, el programa muestra:

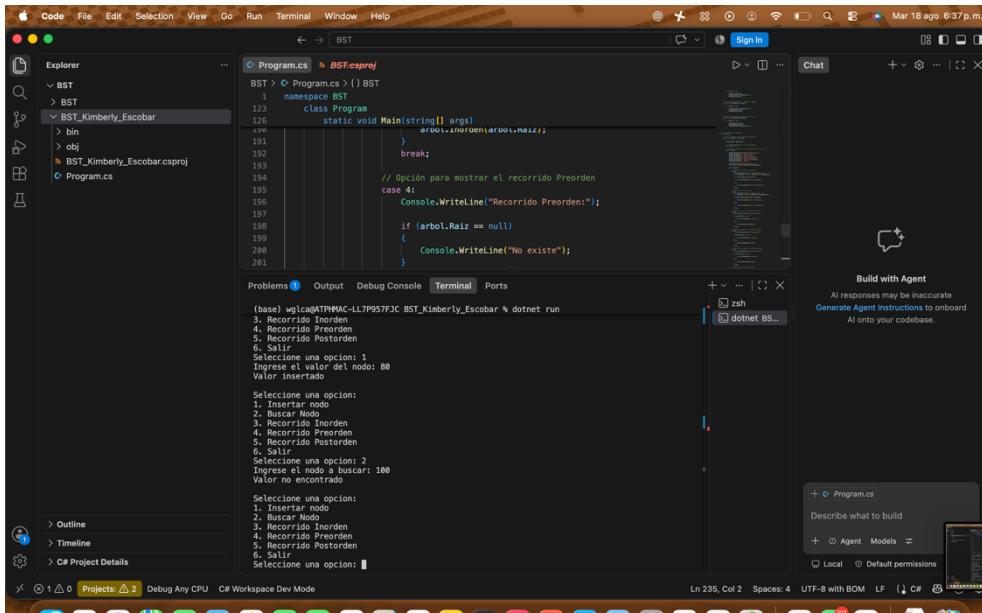
“Valor encontrado”

Si el valor no existe, muestra:

“Valor no encontrado”.



La búsqueda se realiza de forma recursiva directamente sobre el árbol.



5.3 Recorrido Inorden

La opción **3. Recorrido Inorden** muestra los valores utilizando el siguiente orden:

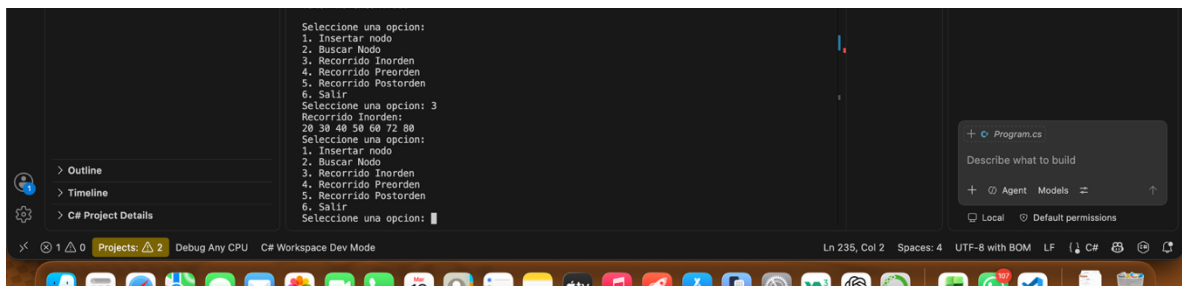
Izquierda → Raíz → Derecha

Para comprobar su funcionamiento se utilizaron los valores:

50, 30, 72, 20, 40, 60, 80

El resultado obtenido fue:

20 30 40 50 60 72 80



El recorrido Inorden permite visualizar los valores del BST ordenados de menor a mayor.

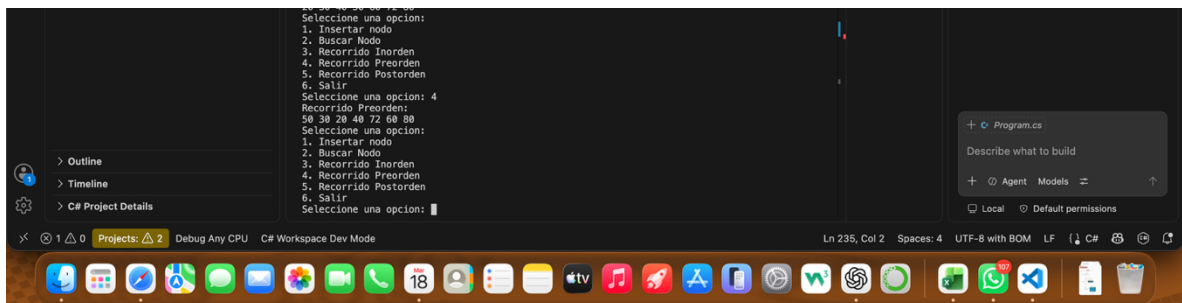
5.4 Recorrido Preorden

La opción **4. Recorrido Preorden** visita los nodos en el siguiente orden:

Raíz → Izquierda → Derecha

Con los valores utilizados en la prueba, el resultado obtenido fue:

50 30 20 40 72 60 80



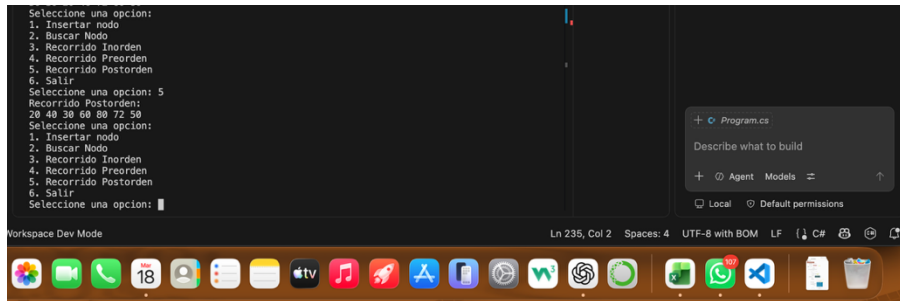
5.5 Recorrido Postorden

La opción **5. Recorrido Postorden** visita los nodos en el siguiente orden:

Izquierda $\rightarrow$ Derecha $\rightarrow$ Raíz

El resultado obtenido durante la prueba fue:

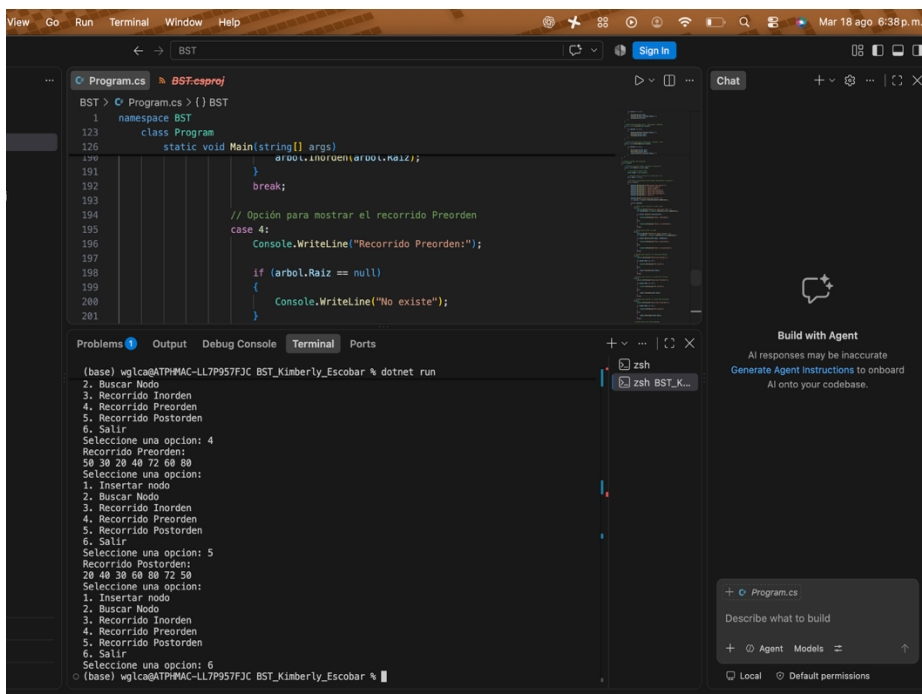
20 40 30 60 80 72 50



5.6 Salir

La opción **6. Salir** permite finalizar la ejecución del programa.

Al seleccionar esta opción, la variable de control del menú cambia su estado y finaliza el ciclo principal.



6. Estructura del programa

El programa está compuesto principalmente por las siguientes clases:

Clase

Nodo

Representa cada elemento almacenado en el árbol.

Contiene:

- `Valor`: almacena el número entero.
- `Izq`: referencia al nodo izquierdo.
- `Der`: referencia al nodo derecho.

Clase

Arbol

Administra la estructura del Árbol Binario de Búsqueda.

Incluye los métodos:

- `Insertar()`
 - `Buscar()`
 - `Inorden()`
 - `Preorden()`
 - `Postorden()`
-
- La inserción, búsqueda y recorridos utilizan recursividad.
 - **Clase**
 - `Program`
 - Contiene el método `Main()` y el menú principal que permite al usuario interactuar con el árbol.

7. Ejemplo de estructura del árbol

Después de insertar los valores:

50, 30, 72, 20, 40, 60, 80

el árbol queda organizado de la siguiente manera:

```

      50
    /   \
   30    72
  / \   / \
 20 40 60 80

```

Esta estructura demuestra cómo los valores menores quedan a la izquierda y los mayores a la derecha.

8. Conclusión

La implementación del Árbol Binario de Búsqueda permitió aplicar conceptos de estructuras de datos, programación orientada a objetos y recursividad en C#. El programa cumple con las operaciones solicitadas, permitiendo insertar y buscar nodos, además de realizar los recorridos Inorden, Preorden y Postorden.

Las pruebas realizadas permitieron comprobar el correcto funcionamiento de cada una de las opciones del menú.

9. Código fuente en GitHub

Repositorio de GitHub:

Repositorio: LINK : <https://github.com/jimenake544930-ai/Laboratorio2>